

Rapporto - Pipeline a due stadi per la previsione dell'errore diagnostico umano

I numeri e le figure si riferiscono al run riproducibile descritto in questo documento.

1. Obiettivo

L'obiettivo è stimare, per ogni decisione di un rater su un caso di risonanza magnetica, quanto è probabile che quella decisione sia sbagliata.

La pipeline ha due stadi collegati:

- **Modello sulle immagini** - dalle tre viste MRI di un caso stima la probabilità che la diagnosi corretta sia positiva (TARGET = 1).
- **Soluzione D** - combina quella probabilità con il rating e il contesto del rater, e stima direttamente la probabilità di errore (error-rating = 1).

Il primo modello non "sa" chi ha valutato il caso; il secondo non vede i pixel, ma solo la probabilità uscita dal primo stadio e le informazioni tabellari. La diagnosi vera (TARGET) non viene usata come variabile di ingresso di D: su un caso nuovo non sarebbe disponibile, quindi usarla in training invaliderebbe l'uso reale.

2. Dati e unità di analisi

Il materiale comprende 427 casi (tre immagini ciascuno: axial, coronal, sagittal) e 13 valutazioni per caso, per un totale di 5551 decisioni caso-rater.

Il file di ground truth ha, per ogni caso, TARGET (0 = diagnosi negativa, 1 = positiva): 236 casi negativi e 191 positivi (prevalenza positiva $\approx 44,7\%$).

Il file delle decisioni ha, per ogni riga, rating, error-rating e variabili di contesto. L'identificativo id ha forma case_id-rater_id.

Abbiamo verificato, unendo i due file sul caso, che su tutte le 5551 righe:

error-rating = 1 se e solo se rating $\neq$ TARGET.

Quindi error-rating = 1 significa proprio "il rater ha sbagliato", non "il caso è positivo". La prevalenza osservata di errori sulle 5551 decisioni è circa 19%; la classe maggioritaria ("nessun errore") è $\sim 81\%$ e va tenuta presente quando si legge l'accuratezza.

Unità di analisi. il modello diagnostico lavora a livello di caso (427). D e i confronti lavorano a livello di decisione (5551), ma la suddivisione in fold e il bootstrap ricampionano i casi: le tredici decisioni dello stesso caso condividono le immagini e non sono indipendenti. Trattarle come 5551 campioni indipendenti restringerebbe in modo falso gli intervalli di confidenza.

3. Definizioni operative

Questa sezione fissa il significato esatto degli oggetti che ricorrono nel rapporto, così che i confronti e i numeri delle sezioni successive siano leggibili senza ambiguità.

3.1 q_{ir} - baseline image-only trasformata sul target error-rating

Per il caso i^* , il modello diagnostico image-only produce:

$$p_i = P(\text{TARGET} = 1 \mid \text{immagini del caso } i)$$

Dato il rating R_{ir} del rater r^* , la baseline è definita così:

se $R_{ir} = 0$, allora $q_{ir} = p_i$

se $R_{ir} = 1$, allora $q_{ir} = 1 - p_i$

q_{ir} è quindi uno score di rischio d'errore costruito da p_i e dal rating osservato, non la probabilità condizionata esatta $P(Y \neq R \mid \text{immagini, rating})$: l'identità richiederebbe che il rating non aggiungesse informazione sulla diagnosi oltre alle immagini, ipotesi smentita dai dati (AUROC del solo rating verso TARGET = 0,81).

Interpretazione. q_{ir} è la probabilità, stimata dal **solo modello image-only** (tramite p_i) **e dal rating del rater**, che quel rater abbia sbagliato. È quindi una baseline ***image-only + rating***: non usa né il contesto del rater né le altre 10 feature di D. È semplice, trasparente e allinea il confronto con D sullo stesso obiettivo.

Verifica della definizione. La formula è stata ricontrollata sulle 5551 decisioni ricostruendola dalle colonne salvate: la differenza massima tra la colonna q_{ir} e la formula applicata a p_i e rating è $1,1 \cdot 10^{-16}$, quindi l'identità è esatta. q_{ir} coincide inoltre, valore per valore, con la feature `ai_probability_rater_wrong`.

Avvertenza necessaria. q_{ir} è uno **score**, non una probabilità calibrata in senso assoluto. Su questo run la media di q_{ir} è **0,3622**, mentre la frequenza osservata di errore è **0,1904**: q_{ir} **sovrastima sistematicamente** la probabilità di errore. È esattamente il motivo per cui q_{ir} , pur essendo il migliore in discriminazione, presenta un Brier Skill Score negativo (vedi paragrafo 6.5).

3.2 Soluzione D

D è un **classificatore tabulare (XGBoost)** addestrato **direttamente su `error-rating`**, con le **11 feature a allowlist chiusa** elencate nel paragrafo 4.4. Le feature combinano il contesto della decisione (rating, confidence, difficoltà, expertise), il profilo del rater ricalcolato fold-safe e cinque feature derivate dal primo stadio (p_i e le sue trasformazioni, fra cui q_{ir}).

TARGET non entra mai come feature di D. È escluso dall'allowlist e protetto da un controllo automatico che solleva errore se TARGET compare tra le colonne del modello. Il motivo è operativo prima che statistico: su un caso nuovo la diagnosi vera non è disponibile, quindi un modello che la usasse in training non sarebbe utilizzabile.

Statistiche del rater. rater-accuracy e rater-confidence non sono i valori globali dell'Excel, ma versioni **fold-safe** ricalcolate per ogni fold (paragrafo 4.4).

3.3 D fast

D fast è **la stessa soluzione D**, ma con le probabilità AI generate da un modello diagnostico addestrato sull'**intero outer-training** e poi **riutilizzato anche per produrre le probabilità dei casi dello stesso outer-training**. Il test outer resta fuori dall'addestramento, ma le probabilità AI usate per addestrare D sul training sono **in parte in-sample**. La definizione 'in-sample' va quindi intesa in senso operativo e non letterale per ogni riga: circa l'80% dell'outer-train contribuisce direttamente al fit, mentre il restante ~20% è usato per l'early stopping. La variante fast è mantenuta come controllo più rapido e potenzialmente ottimistico

Entità dell'ottimismo. Misurata direttamente: le probabilità AI di training nel protocollo fast hanno AUROC **0,9303** rispetto a TARGET, contro **0,7670** nel nested. In fast, D impara quindi da una feature che, in training, è fortemente influenzata dall'aver già visto i casi utilizzati per generarla.

3.4 D nested

D nested è **la stessa soluzione D**, ma le probabilità AI usate per addestrare D sull'outer-training sono ottenute tramite **inner cross-fitting** (4 fold interni sull'outer-train): ogni caso di training riceve una probabilità prodotta da un modello diagnostico **che non lo ha visto**. Per l'outer-test, il modello diagnostico è addestrato **solo** sull'outer-train e poi applicato al test; nell'implementazione si riusano le probabilità diagnostiche outer già fuori-fold, il che è **equivalente** a "fit su outer-train → applica al test", senza il costo di un doppio addestramento.

È quindi la stima **più rigorosa e meno ottimistica** delle prestazioni di D, ed è il protocollo principale di questo rapporto.

Il run documentato è nested. Nei modelli salvati il protocollo è nested, con probabilità di training inner-OOF e probabilità di test outer; la variante fast è stata eseguita sullo stesso codice come controllo e i suoi numeri sono riportati nel paragrafo 6.6.

4. Pipeline implementata

4.1 Validazione, fold e riproducibilità

Prima del training si valida la coerenza dei dati: conteggi, scale, immagini leggibili, relazione error-rating.

I casi sono assegnati a cinque fold esterni stratificati, con un solo fold di test per caso. La stratificazione tiene conto di TARGET e della frequenza media di errore del caso (media di error-rating sui 13 rater, sopra/sotto la mediana), così i fold restano bilanciati sia per la diagnosi sia per la difficoltà empirica. Tutte le 13 decisioni di un caso condividono lo stesso fold: immagini e decisioni dello stesso case_id non finiscono mai in train e test insieme.

Iperparametri e lista chiusa delle feature di D sono fissati in configurazione; seed, snapshot di config e versioni delle dipendenze rendono il run ripetibile.

4.2 Modello sulle immagini

Architettura: ResNet-18 pre-addestrata su ImageNet, condivisa sulle tre viste (axial, coronal, sagittal). Ogni vista produce un embedding 512; i tre vettori si concatenano (1536) e passano a: LayerNorm → linear 1536→256 → ReLU → dropout 0,35 → linear → logit (probabilità via sigmoide).

Preprocessing. Training: RGB, resize 256, random crop 224, rotazione casuale $\pm 5^\circ$, normalizzazione ImageNet. Validation/test: center crop 224, stessa normalizzazione.

Training. Il training è a cinque fold outer; dentro ogni fold outer, ~80% train / ~20% validation stratificati su TARGET. Prime 2 epoche con backbone congelato, poi si aggiorna solo layer4. Early stopping su AUROC di validation (max 15 epoche, patience 4). Il modello migliore predice il test una sola volta.

La soglia operativa usa il criterio di Youden sulla validation: si sceglie la soglia che massimizza sensibilità + specificità - 1 (TPR - FPR), mai ottimizzando sul test.

Il risultato utile per il resto della pipeline è una probabilità fuori-fold per ogni caso: stimata da un modello che non ha visto quel caso in training.

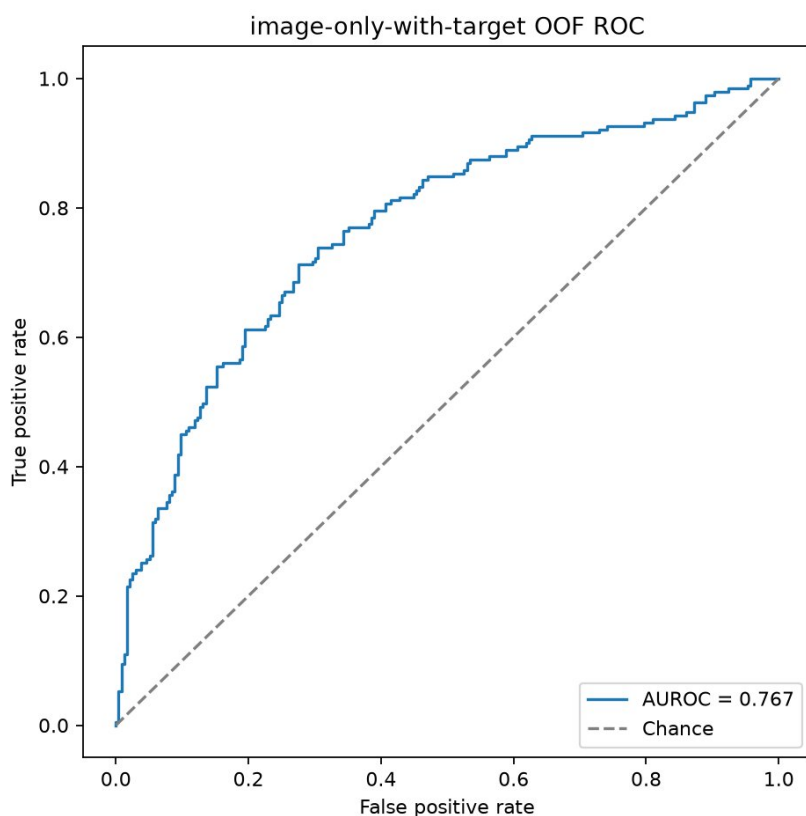


Figura 1. ROC del modello diagnostico (fuori-fold).

Capacità di distinguere diagnosi positive e negative dalle sole immagini.

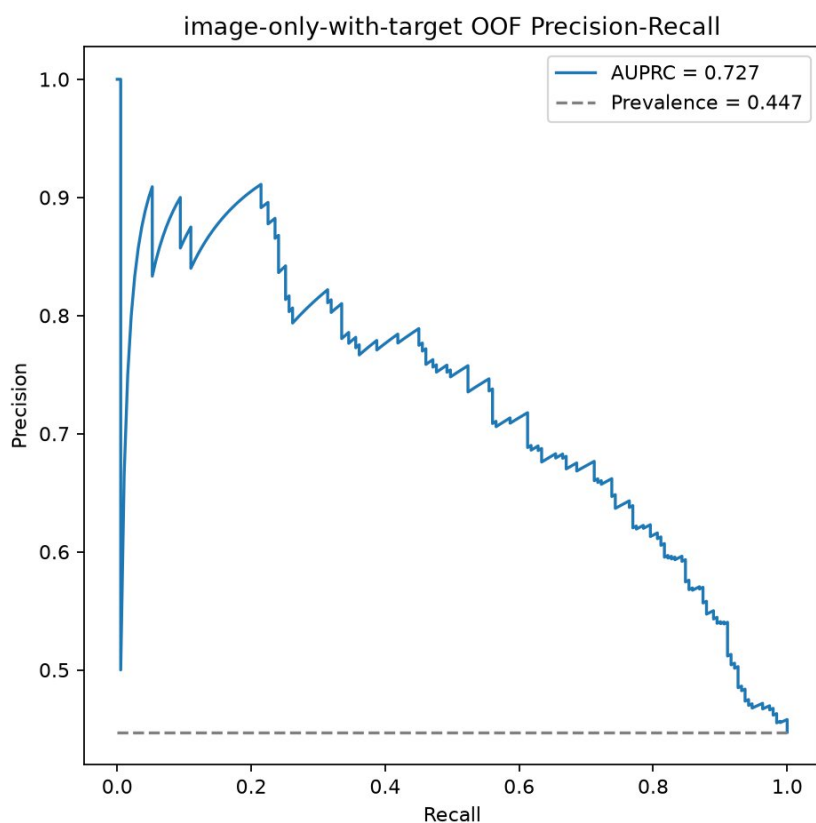


Figura 2.

Precision-recall del modello diagnostico.

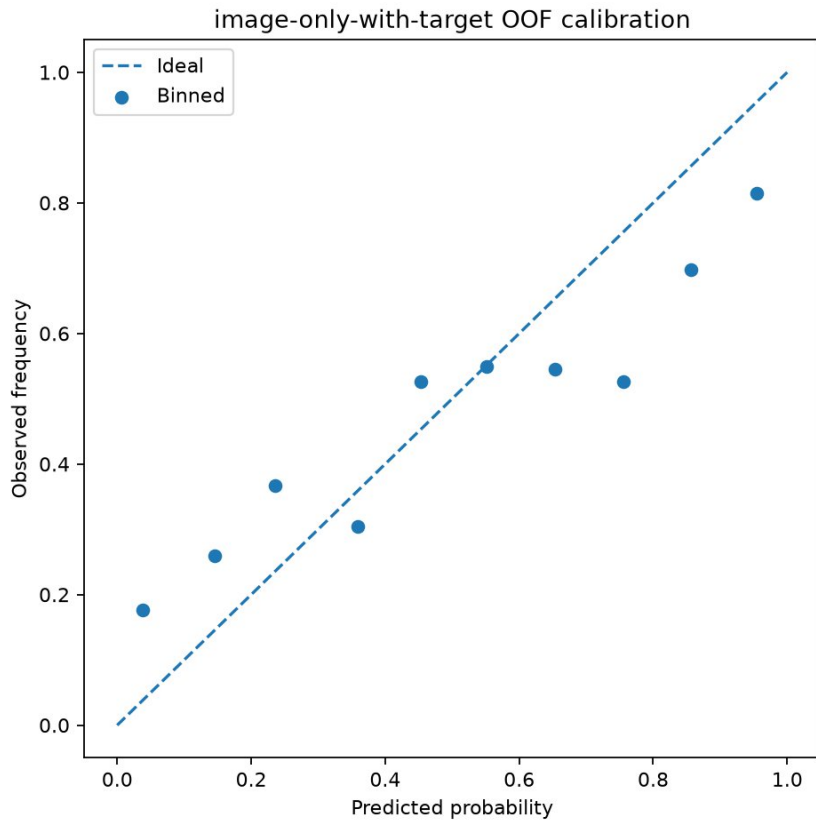


Figura 3. Calibrazione del modello diagnostico.

Quanto le probabilità diagnostiche si avvicinano alle frequenze osservate.

Motivazione delle scelte metodologiche. Serve una probabilità usabile anche in inferenza, senza conoscere TARGET. Congelare e poi sbloccare solo l'ultima parte della rete limita l'adattamento eccessivo a 427 casi. Early stopping e soglia su validation riducono il rischio di adattamento al test.

4.3 Baseline sull'errore: q_{ir}

Per confrontare D in modo lecito serve una baseline sullo stesso obiettivo (error-rating). Confrontare l'AUPRC del diagnostico su TARGET (prevalenza ~45%) con quella di D sull'errore (~19%) mescolerebbe due problemi diversi.

Sia p_i la probabilità che il caso sia positivo e R il rating del rater. Definiamo q_{ir} così (definizione formale e verifica in 3.1):

se $R = 0$, allora $q_{ir} = p_i$;

se $R = 1$, allora $q_{ir} = 1 - p_i$.

Interpretazione: probabilità, basata su immagini e rating, che il rater abbia sbagliato. È semplice, trasparente, e allinea il confronto.

Per ogni decisione si derivano anche cinque feature AI riusate da D: probabilità p_i , classe predetta 1 se $p_i \geq 0,5$ (altrimenti 0), disaccordo con il rating, q_{ir} e margine $|p_i - 0,5|$ (quanto l'AI è "decisa").

4.4 Soluzione D

D è un classificatore tabulare (XGBoost) addestrato a predire error-rating. Usa 11 feature a allowlist chiusa (TARGET escluso):

Origine	Feature
Decisione rater	rating-confidence, case-difficulty, rater-expertise, rating
Profilo rater (fold-safe)	rater-accuracy, rater-confidence
Primo modello	ai_probability_class_1, ai_predicted_class, rater_ai_disagreement, ai_probability_rater_wrong (= q_{ir}), ai_margin

Statistiche rater fold-safe. Nell'Excel, rater-accuracy e rater-confidence sono medie globali su tutto il dataset: usarle in cross-validation leakerebbe il test. Per ogni outer fold si ricalcolano solo sui casi di training, **escludendo l'intero caso della decisione**: poiché ogni rater valuta ogni caso una volta sola, il leave-one-out sulla riga equivale a un leave-one-case-out (differenza misurata **0** su tutti e cinque i fold). Il profilo di una decisione non contiene quindi mai decisioni dello stesso caso, esattamente come in inferenza, dove i profili vengono dai soli casi storici. `rater-accuracy` è inoltre calcolata su `error-rating`, che su un caso nuovo non è disponibile per definizione: le etichette di errore delle altre decisioni sullo stesso caso non entrano mai nel profilo.

Protocollo nested. Il protocollo principale è nested (4 inner fold sul outer-train): sui casi di training esterno le probabilità immagine sono inner-OOF; sui casi di test esterno si usano le probabilità diagnostiche outer già fuori-fold (schema equivalente a "train outer → predici test", senza doppio addestramento). Così D non impara su probabilità in-sample ottimistiche.

Preprocessing e soglia. Preprocessor tabellare per fold: imputazione mediana + standardizzazione fit solo sul train outer. Soglia Youden scelta su validation interna al train outer. Iperparametri XGBoost fissati a priori.

Due varianti. Confrontiamo due varianti (con e senza `scale_pos_weight`): l'analisi principale usa la variante **weighted**; quella **unweighted** serve a mostrare come il peso sulla classe errore sposti recall/specificità/calibrazione, senza scegliere a posteriori la migliore sul test.

Motivazione del protocollo nested e fold-safe. Senza nested, le probabilità AI in-sample gonfierebbero le prestazioni di D. Senza fold-safe, accuratezza/confidence globali introducono leakage dal test. Entrambe le scelte riducono l'ottimismo delle stime.

4.5 Confronto e incertezza

Il confronto ufficiale è appaiato: stesse 5551 decisioni, stesso target, `q_ir` contro D. Si riportano AUROC, AUPRC, Brier, log-loss, indicatori di calibrazione, metriche a soglia 0,5 e curve decisionali descrittive.

Il bootstrap ricampiona i 427 casi (portando sempre tutte le decisioni del caso estratto) e stima intervalli al 95% sulla differenza $\Delta = D - q_{ir}$. Un bootstrap a riga libera sulle 5551 decisioni sottostimerebbe l'incertezza.

4.6 Inferenza su un nuovo caso

L'inferenza usa un ensemble fold-allineato. Per ogni fold $k = 1 \dots 5$: il modello diagnostico del fold k produce `p_i`; si costruiscono le feature (profili rater fold-safe sui casi storici); preprocessor e modello D dello stesso fold stimano $P(\text{error-rating} = 1)$. Alla fine si media sulle cinque probabilità. I componenti dello stesso fold restano abbinati; TARGET non è un input.

```
python -u src/infer_new_case.py --config config.yaml --case-id 1
```

Il comando sopra è una **demo** su un caso già presente nei dati. Per un caso nuovo: tre MRI nel naming previsto, riga nel ground truth e decisioni rater nel file USER, poi `--case-id` dell'identificativo scelto. Prerequisiti, colonne minime, opzioni (`--rater-id`, `--out`) e percorso dell'output JSON sono descritti nella sezione **Inferenza** del README.md; la procedura completa è riassunta nel paragrafo 9.

5. Risultati

5.1 Diagnostico sulle immagini

AUROC fuori-fold $\approx 0,77$; AUPRC $\approx 0,73$. Accuratezza a soglia 0,5 $\approx 0,71$, sopra la baseline di maggioranza ($\sim 0,55$). Il segnale dalle immagini è presente e utile.

Matrici di confusione pooled (427 casi OOF):

Soglia	TN	FP	FN	TP	Accuracy
0,5	173	63	61	130	0,710
Youden (row-wise per fold)	185	51	63	128	0,733

Con soglia Youden row-wise (ogni caso usa la soglia validation del fold in cui era in test; media $\approx 0,541$) l'accuratezza hard sale leggermente; AUROC e Brier restano quelli delle probabilità, indipendenti dalla soglia.

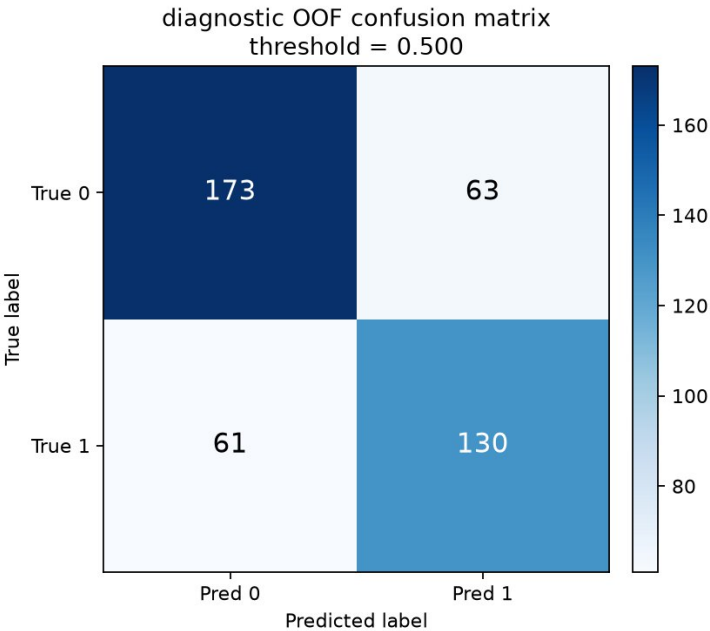


Figura 4. Matrice di confusione diagnostica @0,5.

Classificazione hard pooled OOF a soglia 0,5.

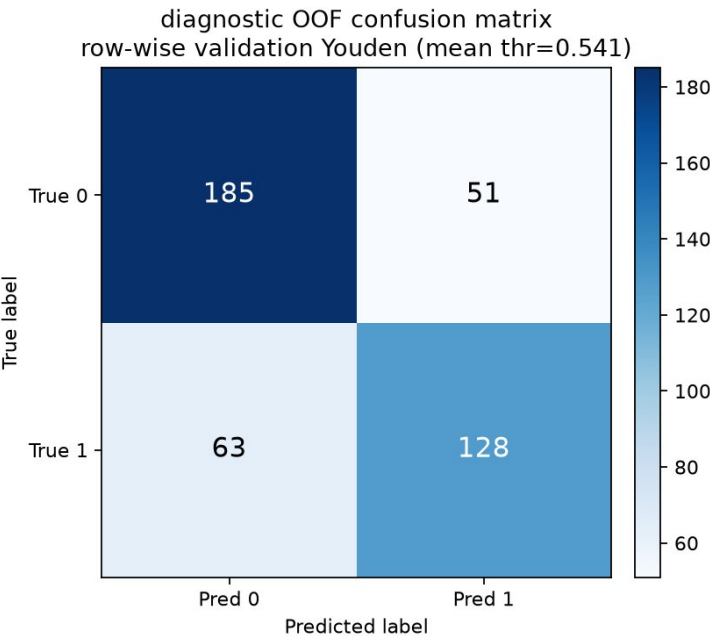


Figura 5. Matrice di confusione diagnostica @Youden.

Stessa valutazione con soglie validation per fold.

5.2 Errore del rater: q_ir contro D

Confronto ufficiale: 5551 decisioni, target error-rating, variante D weighted. I risultati differiscono a seconda della metrica considerata.

Ordinamento (ranking). q_ir presenta un AUROC superiore a D: AUROC $\approx 0,72$ vs $\approx 0,63$; AUPRC $\approx 0,42$ vs $\approx 0,32$. Bootstrap per caso (1000 repliche): Δ AUROC $\approx -0,083$ (IC 95%: $-0,116, -0,051$), intervallo interamente sotto zero $\rightarrow$ vantaggio di q_ir robusto.

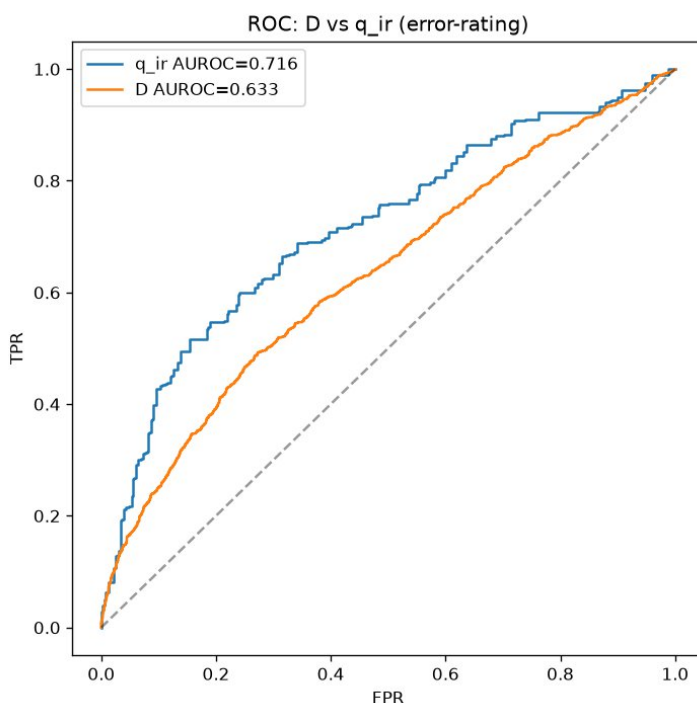
Qualità delle probabilità. D migliora Brier ($\approx 0,152$ vs $0,210$), log-loss ed ECE: Δ Brier $\approx -0,058$ (IC 95%: $-0,083, -0,034$), a favore di D.

A soglia 0,5. D è più conservativa sugli errori: precision $\uparrow$, recall $\downarrow$. L'accuratezza di D ($\sim 0,81$) si avvicina alla classe maggioritaria ($\sim 0,81$ di "non errore"): va letta con balanced accuracy e metriche di ranking.

Punto operativo @0,5 (5551 decisioni):

Modello	Precision	Recall	Specificity	F1	Bal. acc.
q_ir	0,35	0,61	0,73	0,44	0,67
D (weighted)	0,49	0,13	0,97	0,20	0,55

Le due varianti di D. La variante **weighted** di D ha **recall più alto** (0,128 vs 0,032) e **specificity più bassa** (0,969 vs 0,997) rispetto alla **unweighted**, che risulta quindi la più conservativa; la weighted ha però **calibrazione migliore** (ECE 0,065 vs 0,116; Brier 0,152 vs 0,158). Il peso sulla classe errore sposta il trade-off recall/precision/specificità nella direzione attesa — conferma che il peso agisce come previsto — non che D "vinca" in AUROC. Il dettaglio numerico e la verifica del calcolo del peso sono nel paragrafo 6.2 e la tabella affiancata completa nel paragrafo 6.1.



Il confronto ufficiale del rapporto resta quello sulla variante weighted, scelta a priori e non a posteriori sul test.

Figura 6. ROC a confronto su error-rating.

Ordinamento delle decisioni per rischio di errore.

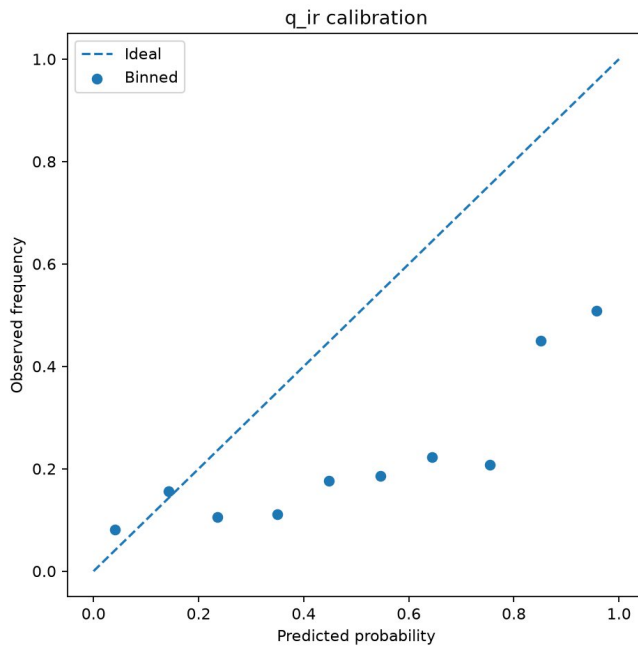


Figura 7. Calibrazione di q_{ir} .

Calibrazione della baseline image-only sull'errore.

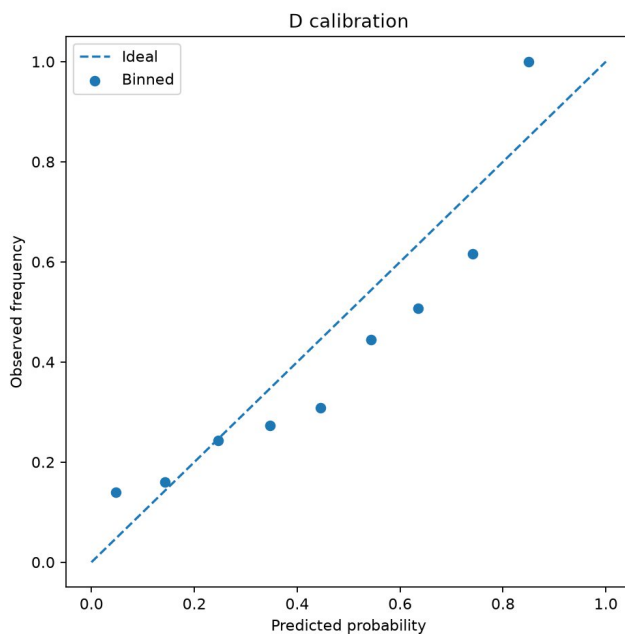


Figura 8. Calibrazione della soluzione D.

Quanto le probabilità di errore di D si avvicinano alle frequenze osservate.

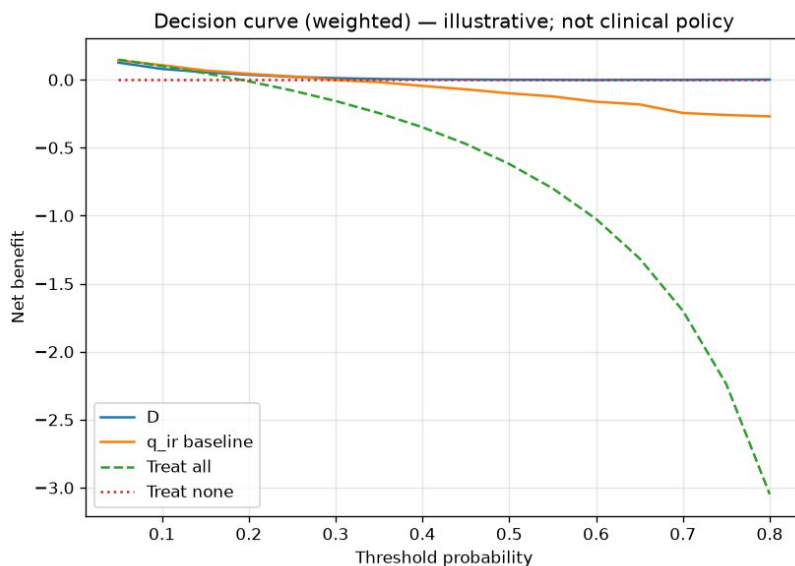


Figura 9. Decision curve (descrittiva).

Beneficio netto al variare della soglia (strumento descrittivo di confronto, non regola decisionale).

5.3 Analisi degli errori (oltre le metriche aggregate)

Oltre al confronto complessivo, abbiamo guardato dove AI, rater e modelli di errore coincidono o si discostano. I conteggi sotto sono sulle 5551 decisioni (e, dove indicato, sui 427 casi).

Accordo tra modello AI diagnostico e rater (su TARGET)

A livello di caso, il modello sulle immagini indovina la diagnosi in 303/427 casi (~71%). A livello di decisione:

Situazione	Conteggio
AI e rater entrambi corretti	3295
AI e rater entrambi errati	413
In disaccordo	1843
– di cui AI corretta, rater sbagliato	644
– di cui AI sbagliata, rater corretto	1199

Restano quindi molti casi di disaccordo: è proprio il terreno in cui ha senso stimare il rischio di errore del rater.

Dove D corregge o peggiora la baseline q_{ir}

Confronto a soglia 0,5 sullo stesso target error-rating (baseline image-only = q_{ir}):

Esito	Conteggio
D corregge un errore di q_{ir}	1073
D sbaglia dove q_{ir} era corretta	521
Entrambi corretti	3418
Entrambi errati	539
Netto (correzioni – peggioramenti)	+552

Il netto sull'accuratezza "hard" è positivo, ma va letto insieme al calo di recall e di balanced accuracy: D indovina più spesso il "non errore", non necessariamente cattura meglio gli errori.

Relazione tra errore, confidence e difficoltà:

rating-confidence	Tasso di errore osservato
1	0,38
2	0,36
3	0,24
4	0,18
5	0,11

case-difficulty	Tasso di errore osservato
1	0,07
2	0,14
3	0,21
4	0,25

Il tasso di errore diminuisce all'aumentare della confidence dichiarata.

Prestazioni di D per expertise e per rater:

Per livello di rater-expertise, l'AUROC di D (weighted, @0,5 sulle probabilità) è circa 0,67 (expertise 2), 0,65 (expertise 3) e 0,62 (expertise 4). Non emerge un guadagno monotonicamente crescente con l'expertise dichiarata.

Per singolo rater (ciascuno valuta tutti i 427 casi) l'AUROC di D varia grosso modo tra ~0,64 e ~0,74. Su alcuni rater il recall a soglia 0,5 è quasi nullo (D predice molto raramente la classe errore): conferma il profilo conservativo già visto in aggregato.

Stabilità fra i cinque fold esterni

AUROC per fold su error-rating:

Fold	q_ir	D
1	0,73	0,66
2	0,79	0,69
3	0,74	0,58
4	0,74	0,68
5	0,65	0,59

In tutti i fold q_ir resta davanti a D in discriminazione; il vantaggio e i livelli assoluti oscillano (fold 3 e 5 più deboli per D). Anche la calibrazione di D resta mediamente migliore (Brier più basso in ciascun fold).

Confronto delle principali metriche

Aspetto	D migliora rispetto a q_ir?
Discriminazione (AUROC / AUPRC)	No - q_ir superiore; non è un effetto "solo di soglia"
Calibrazione / scoring (Brier, ECE)	Sì
Precisione @0,5	Sì (sale)
Recall @0,5	No (scende molto)
Solo soglia operativa?	No - ranking e calibrazione (indipendenti o quasi dalla soglia hard) raccontano già storie opposte

In sintesi: il guadagno principale di D è sulla qualità probabilistica, non sul ranking né sul recupero degli errori a soglia fissa.

6. Verifiche di correttezza

Prima di trarre conclusioni abbiamo verificato esplicitamente i punti da cui dipende la validità dei numeri: che la baseline sia definita come dichiarato, che D impari dal bersaglio giusto, che il peso di classe sia calcolato nella direzione corretta, che le probabilità esportate siano ricostruibili riga per riga, che calibrazione e skill score siano riportati in modo completo e che il protocollo nested non sia contaminato. Ogni verifica è accompagnata dal file o dallo script che la rende riproducibile.

6.1 Metriche pooled delle due varianti di D

Metriche pooled di D **weighted** e D **unweighted** nella stessa tabella. OOF pooled, soglia 0,5, n = 5551 decisioni, prevalenza error-rating = 0,1904.

Metrica	D weighted	D unweighted
AUROC	0,6327	0,6516
AUPRC	0,3247	0,3383
Brier	0,1520	0,1577
log-loss	0,5070	0,5590
Precision (@0,5)	0,4945	0,7234
Recall (@0,5)	0,1277	0,0322
Specificity (@0,5)	0,9693	0,9971
Balanced accuracy (@0,5)	0,5485	0,5146
F1 (@0,5)	0,2030	0,0616
Accuracy (@0,5)	0,8090	0,8134
Baseline di maggioranza (riferimento)	0,8096	0,8096

Lettura. La variante **weighted** ha recall più alto (0,1277 vs 0,0322), precision e specificity più basse, e calibrazione migliore (ECE 0,065 vs 0,116; Brier 0,152 vs 0,158; log-loss 0,507 vs 0,559). La variante **unweighted** è la più conservativa: 34 veri positivi e 1023 falsi negativi su 1057 errori reali, cioè predice quasi mai "errore".

L'**Accuracy** è riportata per completezza ma è la metrica più fuorviante con il 19% di prevalenza: l'accuracy di 0,8090 non supera quella della baseline di maggioranza (0,8096). Per questo il rapporto insiste su balanced accuracy e metriche di ranking.

Fonte: outputs/compare/unified_D_metrics_table.csv,
outputs/D/metrics_pooled_weighted.csv,
outputs/D/metrics_pooled_unweighted.csv.

6.2 scale_pos_weight: calcolo per fold e verifica della direzione

Documentiamo per ciascun fold il numero di positivi, il numero di negativi e il valore effettivo di `scale_pos_weight`, verificando che sia stato usato il rapporto **n_negative / n_positive** e non il reciproco.

Fold	n positivi (error-rating=1)	n negativi	scale_pos_weight	Formula usata	n_neg/n_pos	n_pos/n_neg (reciproco)
1	874	3559	4,072082	n_negative / n_positive	4,072082	0,245575
2	863	3570	4,136732	n_negative / n_positive	4,136732	0,241737
3	864	3582	4,145833	n_negative / n_positive	4,145833	0,241206
4	792	3654	4,613636	n_negative / n_positive	4,613636	0,216749
5	835	3611	4,324551	n_negative / n_positive	4,324551	0,231238

I conteggi sono calcolati su **tutte le righe di outer-train del fold** (4433 righe al fold 1 = 874 + 3559), cioè esattamente sull'insieme usato per addestrare il modello.

Tre verifiche indipendenti.

1. **Confronto numerico.** Per ogni fold il valore di `scale_pos_weight` coincide con `n_negative / n_positive` (match esatto) e non con il reciproco.
2. **Valore effettivamente memorizzato nei modelli.** Nei cinque modelli salvati il valore registrato nel payload, quello restituito dai parametri del classificatore e l'attributo interno del modello valgono **4,072082 / 4,136732 / 4,145833 / 4,613636 / 4,324551**, cioè esattamente i rapporti `n_neg/n_pos`. Nelle varianti `unweighted` l'attributo è assente (nessun peso applicato).
3. **Controprova sperimentale.** Rifittando il fold 1 con lo stesso codice, le stesse feature e lo stesso seed, cambiando **solo** `scale_pos_weight`:

scale_pos_weight	Significato	AUROC	Recall @0,5	Precision @0,5	Specificity @0,5	media prob. predetta
4,072082	usato nel run (n_neg/n_pos)	0,6599	0,1257	0,5349	0,9786	0,1943
1,000000	nessun peso	0,6903	0,0055	1,0000	1,0000	0,0879
0,245575	reciproco (n_pos/n_neg)	0,6939	0,0000	0,0000	1,0000	0,0348

La monotonia è quella attesa e nella direzione corretta: aumentando il peso sulla classe positiva, recall e probabilità media predetta **salgono**, precision e specificity **scendono**. Se fosse stato usato il reciproco, la variante "weighted" sarebbe stata ancora più conservativa della unweighted: l'opposto di quanto si osserva nel paragrafo 5.2.

Come controllo di riproducibilità, rifittando il fold 1 con il peso documentato 4,072082 si ritrovano le probabilità OOF salvate con differenza massima **$2,9 \cdot 10^{-8}$** (del tutto trascurabile) e AUROC di fold identica (0,6599).

Conclusione. Il calcolo è corretto: $\text{scale_pos_weight} = n_negative / n_positive$, pari a $\approx 4,07-4,61$, sempre > 1 , quindi il peso va nella direzione giusta, cioè rinforza la classe minoritaria "errore". La differenza osservata nella variante weighted non ha quindi origine nel calcolo del peso, ed è spiegata da quanto riportato nel paragrafo 5.2.

Fonte: outputs/D/scale_pos_weight_per_fold.csv.

6.3 predict_proba[:, 1] come probabilità della classe "errore"

Verifichiamo esplicitamente che, in ogni modello D addestrato, `predict_proba(X)[: , 1]` corrisponda alla probabilità della classe `error-rating = 1`, cioè la classe "errore". Da questo dipendono le soglie Youden e tutte le metriche di D.

Verifica dell'ordine delle classi. XGBoost ordina le colonne di `predict_proba` secondo `classes_`. Poiché le etichette sono binarie $\{0, 1\}$, la colonna 1 è $P(\text{error-rating} = 1)$ **se e solo se** `classes_ == [0, 1]`.

Esito: verificato su tutti i 20 modelli salvati ($5 \text{ fold} \times \{\text{nested, fast}\} \times \{\text{weighted, unweighted}\}$).

4. **Ordine delle classi.** Per ogni modello salvato: `n_classes_ == 2` e `classes_ == [0, 1]`. Verifica registrata nell'artefatto `outputs/D/predict_proba_class_verification.csv` (una riga per modello, con esito positivo), generato da uno script di controllo apposito.
`tmp/verify_predict_proba_class_order.py`
5. **Guardrail a runtime nel training.** Il training esegue il controllo `classes_ == [0, 1]` sia dopo il fit del modello usato per la soglia sia dopo il fit del modello finale, subito prima di usare `[: , 1]`: un futuro run con ordine di classi diverso **fallisce** invece di produrre numeri sbagliati in silenzio.
6. **Verifica semantica della colonna 1.** Sul modello di fold 1 salvato: la sigmoide del margine grezzo coincide con `predict_proba[: , 1]` (identità esatta), le righe di `predict_proba` sommano a 1 e l'`argmax` coincide con `predict`.
7. **Controprova empirica sulla direzione del segnale.** Se la colonna 1 fosse la probabilità della classe complementare, il segno della relazione con `error-rating` si invertirebbe:

Quantità	Valore	Significato
<code>AUROC(d_probability, error-rating)</code>	0,6327	$> 0,5 \rightarrow$ valore alto $\Rightarrow$ più probabile l'errore
<code>AUROC(1 - d_probability, error-rating)</code>	0,3673	è ciò che si otterrebbe con la colonna invertita
<code>correlazione(d_probability, error-rating)</code>	+0,2218	segno positivo, coerente con la classe "errore"

Conclusione. `predict_proba[:, 1]` è la probabilità della classe "errore" in ogni modello addestrato (weighted e unweighted, nested e fast); le soglie e le metriche di D si basano quindi sulla probabilità corretta.

6.4 Predizioni OOF riga per riga

Per rendere ricostruibile il confronto appaiato senza ambiguità, le predizioni fuori-fold sono esportate riga per riga sia per `q_ir` sia per `D`, con almeno `case_id`, `rater_id`, `y_true`, probabilità predetta e `fold`. I file sono rigenerati automaticamente ad ogni run della pipeline.

Formato comune (6 colonne, 5551 righe, fold 1...5):

`id, case_id, rater_id, y_true, predicted_probability, fold`

File	Modello	y_true	predicted_probability	Righe	Intervallo probabilità
outputs/baseline/oof_q_ir_predictions.csv	baseline q_ir	error-rating	q_ir	5551	0,0002 – 0,9998
outputs/D/oof_D_predictions.csv	D nested weighted	error-rating	d_probability	5551	0,0039 – 0,8895
outputs/D/oof_D_predictions_unweighted.csv	D nested unweighted	error-rating	d_probability	5551	0,0024 – 0,7504
outputs/D/oof_D_predictions_fast.csv	D fast weighted	error-rating	d_probability	5551	0,0006 – 0,9887
outputs/D/oof_D_predictions_unweighted_fast.csv	D fast unweighted	error-rating	d_probability	5551	0,0005 – 0,9776

Garanzie di qualità applicate dall'export (e verificate): nessun `id` duplicato, `y_true` binario {0, 1}, probabilità finite, righe ordinate per `case_id` e `rater_id`; ogni file copre i 427 casi × 13 rater, con fold 1...5.

Utilità degli export OOF. Con questi file il confronto `q_ir` vs `D` è riproducibile senza riaddestrare: basta unire su `id` e usare `y_true` come etichetta. Le probabilità di `q_ir` e di `D` sono allineate sulla stessa riga decisione (`id = case_id-rater_id`) e sullo stesso `fold`, quindi il confronto è appaiato per costruzione; il bootstrap va comunque eseguito **per `case\_id`**, portando tutte e 13 le decisioni del caso insieme (paragrafo 4.5).

Fonte: `src/export_oof_predictions.py`, agganciato a `src/build_baseline.py` e a `src/train_D.py`.

6.5 Calibrazione (slope, intercept), Brier, ECE e Brier Skill Score

Completiamo il quadro delle probabilità con calibration intercept e calibration slope per q_{ir} e per D, oltre a Brier ed ECE, e con il **Brier Skill Score** rispetto alla baseline costante data dalla prevalenza di error-rating.

OOF pooled, soglia 0,5, $n = 5551$, prevalenza = 0,1904. Il Brier della **baseline costante** (predire sempre la prevalenza) è **0,154158**.

Modello	AUROC	AUPRC	Brier	BSS	log-loss	calibration slope	calibration intercept	ECE	media $\hat{p}$
q_{ir} (baseline image-only)	0,7159	0,4158	0,2101	-0,3629	0,6914	0,3814	0,0523	0,2015	0,3622
D weighted	0,6327	0,3247	0,1520	+0,0139	0,5070	0,5674	0,1006	0,0653	0,1582
D unweighted	0,6516	0,3383	0,1577	-0,0230	0,5590	1,0616	0,1088	0,1160	0,0769

Definizione del Brier Skill Score usata:

$$\text{BSS} = 1 - \text{Brier}(\text{modello}) / \text{Brier}(\text{baseline costante})$$

$$\text{Brier}(\text{baseline costante}) = \bar{p} \cdot (1 - \bar{p}) \quad \text{con } \bar{p} = \text{prevalenza di error-rating}$$

BSS = 0 significa equivalenza a predire sempre la prevalenza; BSS > 0 meglio della costante; BSS < 0 peggio della costante.

Lettura.

- **`q<sub>ir</sub>` ha BSS negativo (-0,3629):** pur essendo il migliore in discriminazione (AUROC 0,7159), come *probabilità* è peggiore di una costante pari alla prevalenza. Il motivo è visibile nella colonna "media $\hat{p}$ ": q_{ir} predice in media 0,3622 contro una frequenza reale di 0,1904, quindi **sovrastima sistematicamente l'errore**. Lo slope 0,3814 $\ll$ 1 conferma che questi valori non vanno letti come percentuali cliniche.
- **D weighted è l'unico con BSS positivo (+0,0139)**, con ECE molto più basso (0,0653 vs 0,2015) e log-loss 0,5070 vs 0,6914: come *score probabilistico* è il migliore dei tre. Il guadagno rispetto alla baseline costante è però **marginale**.
- **D unweighted** ha lo slope più vicino all'ideale (1,0616) ma ECE peggiore (0,1160) e media $\hat{p}$ 0,0769: predice sistematicamente meno errori di quanti ne accadono, coerente con il profilo conservativo visto nel recall.

Nota metodologica. Slope $\approx$ 1 e intercept $\approx$ 0 sono l'ideale; qui nessuno dei tre modelli è ben calibrato in senso assoluto. Brier, ECE, slope e intercept vanno letti come **diagnostica di calibrazione**, non come correzione: nessuna calibrazione post-hoc è stata applicata, per scelta di scope.

Il Brier Skill Score è riportato anche con incertezza: nel bootstrap appaiato per caso compare come riga dedicata in `outputs/compare/{weighted,unweighted}/bootstrap_paired_deltas.csv` (stima per q_{ir} e per D, $\Delta = D - q_{ir}$, IC 95%), oltre che in `outputs/compare/compare_summary.json`.

Fonte: outputs/baseline/baseline_q_ir_metrics.csv,
 outputs/D/metrics_pooled_weighted.csv,
 outputs/D/metrics_pooled_unweighted.csv,
 outputs/compare/{weighted,unweighted}/bootstrap_paired_deltas.csv.

6.6 Controllo del protocollo: D nested contro D fast

Il controllo più importante riguarda il protocollo. Sullo **stesso codice**, con gli stessi fold, le stesse 11 feature, la stessa selezione della soglia, gli stessi iperparametri XGBoost, lo stesso seed e lo stesso preprocessing, abbiamo eseguito anche la versione **fast**, lasciando invariato tutto il resto. L'unica differenza è la provenienza delle probabilità AI usate per addestrare D (paragrafo 3.3 e paragrafo 3.4).

Risultato pooled (D weighted, OOF, n = 5551):

Protocollo	AUROC	AUPRC	Brier	log-loss
D nested	0,6327	0,3247	0,1520	0,5070
D fast	0,7028	0,3888	0,1556	0,5518
Δ (fast – nested)	+0,0701	+0,0640	+0,0036	+0,0448

Bootstrap appaiato **per caso** (1000 repliche, ricampionando i 427 casi con tutte le loro decisioni):

Δ AUROC (fast – nested) = +0,0707 IC 95% [+0,0339 ; +0,1074] → interamente sopra zero

La differenza non dipende da un fold fortunato: il fast ha AUROC maggiore in tutti e cinque i fold

Fold	AUROC nested	AUROC fast	Δ
1	0,6599	0,7397	+0,0798
2	0,6948	0,7032	+0,0084
3	0,5752	0,7413	+0,1662
4	0,6762	0,6954	+0,0192
5	0,5867	0,6754	+0,0887

Anche per la variante unweighted la differenza di AUROC va nella stessa direzione: nested 0,6516 vs fast 0,7097. Il confronto ufficiale del rapporto resta comunque quello sulla variante weighted.

Confrontiamo la qualità delle probabilità AI che D usa in **training**, misurata rispetto a TARGET:

Probabilità AI	Provenienza	AUROC vs TARGET
training nested	inner-OOF (casi mai visti dal modello che le produce)	0,7670
training fast	in-sample (il modello ha visto i casi)	0,9303
test nested	OOF outer	0,7671
test fast	out-of-sample	0,7794

Le probabilità AI di training nel protocollo fast sono quasi un **indicatore dell'etichetta del caso stesso** (AUROC 0,9303 contro 0,7670). D impara quindi una dipendenza dalle feature AI molto più forte e "pulita" di quella che può esistere all'uso reale, dove per un caso nuovo la probabilità AI è necessariamente out-of-sample (AUROC $\approx$ 0,77). È esattamente l'ottimismo che il protocollo nested evita: **non** un leakage delle etichette di test nel training, ma un **disallineamento tra la feature AI vista in training e quella disponibile su casi nuovi**.

Questo è coerente con il fatto che, nel protocollo fast, il ranking migliori (AUROC e AUPRC su) mentre la qualità probabilistica peggiora (Brier 0,1556 vs 0,1520; log-loss 0,5518 vs 0,5070).

Interpretazione. A parità di codice, l'unica cosa che cambia è la provenienza di `p_i`, e la differenza di AUROC che ne deriva (+0,07) spiega il divario tra una stima di circa 0,70 e una di circa 0,63. Un AUROC vicino a 0,70 su questo task è quindi il valore atteso quando D impara da probabilità AI in parte in-sample; la stima nested ($\approx$ 0,63) è quella più coerente con il protocollo previsto per l'utilizzo su casi nuovi, ed è per questo che è il protocollo ufficiale del rapporto. Questo controllo quantifica inoltre, con un numero, l'affermazione qualitativa della tabella delle decisioni progettuali ("fast tende a sovrastimare le prestazioni", paragrafo 8).

Fonte: `outputs/compare/nested_vs_fast_comparison.csv`,
`outputs/D/fast_probs_fold_{1..5}.csv`,
`outputs/D/oof_predictions{,_fast}.csv`,
`outputs/D/metrics_pooled_weighted{,_fast}.csv`,
`models/D/scenario_D_fold*_fast.joblib`.

7. Discussione e validità delle stime

L'analisi integrata dei due stadi rivela un comportamento complementare tra la baseline q_{ir} e la soluzione D.

Riguardo al bilanciamento tra discriminazione e calibrazione, sebbene l'inclusione delle variabili contestuali del rater non migliori la capacità di ordinare le decisioni per rischio di errore (AUROC pari a 0,72 per q_{ir} contro 0,63 per D), il modello D mostra una qualità probabilistica migliore rispetto a q_{ir} (Brier 0,152 contro 0,210; log-loss ed ECE inferiori), con un BSS solo leggermente positivo (+0,0139). La baseline q_{ir} , pur essendo un eccellente predittore ordinale, sovrastima sistematicamente l'errore medio, registrando un valore stimato di 0,36 contro un valore reale di 0,19.

Sul piano delle implicazioni operative, alla soglia di 0,5 il modello D adotta un profilo altamente conservativo, caratterizzato da un'elevata specificità pari a 0,97 e da un basso recall pari a 0,13. Di conseguenza, la soluzione D è preferibile quando l'obiettivo è produrre probabilità ben tarate per un supporto decisionale quantitativo, mentre q_{ir} risulta superiore per lo screening o il ranking dei casi a maggior rischio di errore umano. L'attendibilità delle stime risiede nelle garanzie metodologiche adottate per evitare errori di valutazione, in particolare nel rigoroso isolamento delle informazioni. Questo include l'assenza della variabile TARGET tra le feature di D, il ricalcolo fold-safe delle statistiche dei rater e l'adozione del protocollo nested cross-fitting. Come dimostrato quantitativamente dalla differenza di AUROC pari a +0,07 riscontrata nel protocollo fast, l'uso di probabilità in-sample avrebbe introdotto un forte ottimismo ingiustificato. Infine, la validità dei risultati deve tenere conto di alcuni limiti dello studio. In primo luogo, la presenza di un singolo dataset comporta la mancanza di una coorte di test esterna per valutare la generalizzabilità a nuovi centri o rater. In secondo luogo, non sono state applicate tecniche di calibrazione post-hoc, come il Platt scaling o la regressione isotonica. Infine, va considerato il costo computazionale, in quanto la procedura nested richiede tempi di calcolo elevati, pur essendo indispensabile per garantire stime non contaminate.

7.1 Errori metodologici evitati

La pipeline è stata progettata per ridurre i principali rischi di sovrastima delle prestazioni e di perdita di comparabilità tra le soluzioni. In particolare, sono stati evitati l'utilizzo di TARGET come feature del modello D, il calcolo di statistiche del rater utilizzando informazioni provenienti dal test, la sovrapposizione dello stesso caso tra train e test e la scelta delle soglie direttamente sul test.

Sono stati inoltre mantenuti distinti il processo di addestramento e quello di valutazione delle probabilità AI, utilizzando una procedura nested per D, e l'incertezza è stata stimata a livello di case_id per tenere conto della dipendenza tra le decisioni riferite allo stesso caso. Il confronto tra q_{ir} e D è stato effettuato sullo stesso target e con metriche calcolate secondo un protocollo comune.

Le verifiche del paragrafo 6 documentano i controlli specifici alla base di queste scelte, inclusi l'ordine delle classi utilizzato da predict_proba, il calcolo di scale_pos_weight, la produzione delle predizioni OOF e il confronto tra D nested e D fast. In questo modo, le scelte metodologiche non restano assunzioni implicite, ma sono accompagnate da verifiche riproducibili.

Le probabilità prodotte dai modelli devono infine essere interpretate come score predittivi. La loro affidabilità come frequenze osservate va valutata attraverso le curve e le metriche di calibrazione; analogamente, la decision curve analysis ha finalità descrittiva e non definisce una regola decisionale clinica.

8. Decisioni progettuali e alternative

Decisione	Scelta	Alternative considerate	Effetto sulla validità
Batch size diagnostico	12	8–10	Nel range consentito; bilancia rumore e VRAM su n=427
Aggregazione viste	Concat 512×3	Media embedding; tre reti separate	Preserva informazione per vista senza triplicare parametri
Protocollo probabilità AI per D	Nested (4 inner fold)	Fast (in-sample su train)	Nested evita ottimismo; fast tende a sovrastimare le prestazioni (quantificato nel paragrafo 6.6: AUROC $\approx$ 0,70 fast vs $\approx$ 0,63 nested)
Probabilità AI su outer-test	Riuso OOF diagnostiche outer	Rifit diagnostico su tutto outer-train	Schema equivalente; evita costo doppio
Statistiche rater in CV	Fold-safe (LOO / train-only)	Medie globali Excel	Le medie globali leakano il test
Confronto con D	Baseline q _{ir} su error-rating	AUPRC grezza su TARGET	Allinea target e prevalenza
Bootstrap	Per case _{id}	Per riga sulle 5551	Rispetta la dipendenza intra-caso
Soglia operativa	Youden su validation	Tuning sul test	Non adatta la soglia al test
Iperparametri	Fissi in config	Search sul test	Evita selezione ottimistica
Freeze rete	2 epoche + solo layer4	Fine-tune completo subito	Meno overfitting su n=427
Weighting errori	Weighted e unweighted	Una sola variante	Consente di discutere il trade-off senza selezionare a posteriori la variante più favorevole
Peso di classe	scale_pos_weight = n _{neg} /n _{pos}	Reciproco n _{pos} /n _{neg} ; nessun peso	Direzione corretta e verificata (paragrafo 6.2); il reciproco non è stato usato
Probabilità di errore usata	Colonna 1 di predict_proba	Colonna 0 (complemento)	Verificata come P(error-rating=1) su tutti i modelli (paragrafo 6.3)

9. Conclusioni

In questo lavoro è stata sviluppata e valutata una pipeline a due stadi per la stima del rischio di errore diagnostico umano su immagini MRI.

Nel primo stadio, il modello sulle immagini dimostra di estrarre un segnale diagnostico solido, raggiungendo un AUROC di circa 0,77. Nel secondo stadio, focalizzato sul target di errore diagnostico, si osserva che non esiste una soluzione dominatore assoluta. La baseline q_{ir} prevale nella capacità di ordinamento delle decisioni con un AUROC di circa 0,72, mentre la soluzione tabellare D mostra una qualità probabilistica migliore (Brier ed ECE più bassi) e un Brier Skill Score solo leggermente positivo. Sotto il profilo del rigore metodologico, l'adozione di un protocollo nested si è dimostrata un elemento essenziale per prevenire sovrastime ottimistiche delle prestazioni.

In conclusione, i risultati documentano un trade-off tra discriminazione e qualità probabilistica: q_{ir} ordina meglio le decisioni, D produce probabilità meno distanti dalle frequenze osservate, senza raggiungere una calibrazione assoluta soddisfacente. Con un solo dataset e nessuna coorte esterna non si può stabilire quale soluzione sia preferibile in un workflow clinico. Gli sviluppi futuri sono potrebbero incentrarsi sulla calibrazione post-hoc.

Appendice A - Riproducibilità: comandi e artefatti

Setup. Python 3.11 (verificato con 3.11.9); dipendenze in `requirements.txt`. Le versioni esatte del run sono registrate in `outputs/environment.txt` e la configurazione completa in `outputs/run_config_snapshot.yaml`, così il run è ricostruibile.

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Per l'addestramento su GPU conviene installare PyTorch dal sito ufficiale con il comando CUDA adatto al sistema, poi le altre dipendenze; il training diagnostico e il nested di D sono lunghi e beneficiano di una GPU.

Ordine della pipeline. Ogni passo usa gli artefatti del precedente; eseguirli in quest'ordine.

```
python -u src/validate_dataset.py --config config.yaml
```

```
python -u src/generate_folds.py --config config.yaml
```

```
python -u src/train_diagnostic.py --config config.yaml      # lungo; GPU consigliata
```

```
python -u src/build_baseline.py --config config.yaml
```

```
python -u src/build_D_features.py --config config.yaml --smoke # opzionale
```

```
python -u src/train_D.py --config config.yaml --protocol nested # molto lungo
```

```
python -u src/compare_D_vs_baseline.py --config config.yaml --n-bootstrap 1000
```

```
python -u src/analyze_errors.py --config config.yaml
```

All'avvio degli script principali si aggiornano `outputs/run_config_snapshot.yaml` e `outputs/environment.txt`.

Inferenza su un caso (procedura completa nel paragrafo 4.6):

```
python -u src/infer_new_case.py --config config.yaml --case-id 1
```

```
# opzionale: --rater-id 0
```

```
# opzionale: --out path/to/out.json
```

Artefatti principali:

Path	Contenuto
outputs/folds/case_fold_assignment.csv, fold_balance_stats.csv	assegnazione dei fold e bilanciamento per fold
outputs/diagnostic/oof_predictions.csv	probabilità fuori-fold del primo stadio per i 427 casi
outputs/diagnostic/metrics_per_fold.csv, metrics_pooled.csv	metriche del primo stadio, per fold e pooled
outputs/diagnostic/roc_oof.png, pr_oof.png, calibration_oof.png	curve ROC, PR e calibrazione del primo stadio
outputs/diagnostic/confusion_matrix_oof.csv, confusion_matrix_oof@*.png	matrici di confusione pooled
outputs/baseline/decision_level_features.csv	tabella a livello decisione con q _{ir} e le 5 feature AI
outputs/baseline/oof_q _{ir} _predictions.csv, baseline_q _{ir} _metrics.csv	predizioni q _{ir} riga per riga e metriche
outputs/baseline/roc_q _{ir} .png, pr_q _{ir} .png, calibration_q _{ir} .png	curve della baseline
outputs/D/oof_predictions{, _unweighted}{, _fast}.csv	predizioni OOF di D (varianti e protocolli)
outputs/D/oof_D_predictions{, _unweighted}{, _fast}.csv	export puliti di D riga per riga (paragrafo 6.4)
outputs/D/metrics_per_fold_*.csv, metrics_pooled_*.csv	metriche di D per fold e pooled
outputs/D/scale_pos_weight_per_fold{, _fast}.csv	peso di classe per fold (paragrafo 6.2)
outputs/D/predict_proba_class_verification.csv	verifica dell'ordine delle classi (paragrafo 6.3)
outputs/D/nested_inner_oof_fold_{1..5}.csv	probabilità inner-OOF usate da D in training
outputs/D/rater_profiles_foldsafe_fold_{1..5}.csv	profili rater fold-safe per fold
outputs/compare/{weighted, unweighted}/	ROC/PR/calibrazione affiancate, decision curve, metrics_point.csv, bootstrap_paired_deltas.csv
outputs/compare/compare_summary.json	riepilogo del confronto, con BSS
outputs/compare/unified_D_metrics_table.csv	tabella affiancata delle due varianti (paragrafo 6.1)
outputs/compare/nested_vs_fast_comparison.csv	confronto dei protocolli (paragrafo 6.6)
outputs/analysis/analysis_summary.json	accordo AI-rater, correzioni di D su q _{ir} , lettura dei sottogruppi
outputs/analysis/metrics_by_*.csv, error_rate_by_*.csv, fold_stability_metrics.csv	metriche per sottogruppo e stabilità per fold
outputs/analysis/infer_case_{id}.json	output dell'inferenza su un caso
models/diagnostic/diagnostic_fold_{1..5}.pt	checkpoint del primo stadio
models/D/scenario_D_fold_*.joblib, preprocessor_fold_*.joblib	modelli D e preprocessori per fold: le coppie dello stesso fold vanno usate insieme
outputs/run_config_snapshot.yaml, outputs/environment.txt	configurazione e versioni del run

Nota sull'inferenza. I prerequisiti sono: setup completato; presenti `models/diagnostic/diagnostic_fold_1..5.pt`, `models/D/scenario_D_fold_*.joblib`, `models/D/preprocessor_fold_*.joblib` e `outputs/folds/case_fold_assignment.csv`. Per ogni fold k il modello diagnostico k produce p_i , le feature vengono costruite con i profili rater fold-safe dei casi storici, e preprocessor k con modello D_k restituiscono una probabilità di errore; le cinque probabilità vengono poi mediate. TARGET non è un input di D . La demo con `--case-id` di un caso già presente nei dati serve a verificare il cablaggio e non è una stima di prestazione, perché parte dei diagnostici ha visto quel caso in addestramento.

Moduli di supporto (non si lanciano da soli, contengono solo funzioni ecc...): `data.py` (caricamento Excel, tabella casi, dataset immagini), `models.py` (architettura DiagnosticNet), `metrics.py` (metriche binarie, calibrazione, BSS, plot), `methodology_guards.py` (allowlist di D , statistiche rater fold-safe, soglie su validation, bootstrap per caso, controllo di disgiunzione dei casi), `reproducibility.py` (seed e snapshot).